

ShopSphere

Full-Scale Amazon/Flipkart-Style Microservices Platform — Complete Learning Roadmap

Yeh extended roadmap ek real e-commerce platform (Amazon/Flipkart-scale) banate hue Microservices, distributed systems patterns aur har industry-standard tool seekhne ke liye hai — auth se lekar search, notifications, invoices, seller dashboard, sales analytics, SSO/OAuth aur Git/DevOps practices tak, sab kuchh real problems solve karte hue.

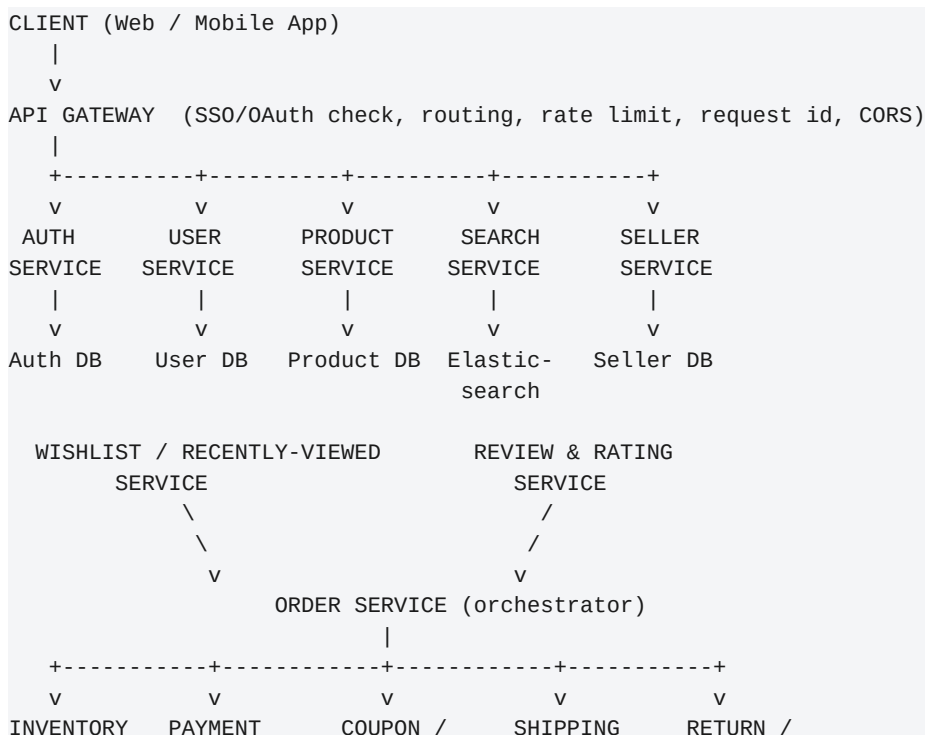
24 Phases	20+ Services / Components	10+ Distributed-System Patterns	Python / FastAPI Core Stack
---------------------	-------------------------------------	---	---------------------------------------

Core Learning Principle

Har technology sirf 'use karne ke liye' nahi seekhi jayegi — pehle ek real production problem create hogi, phir uska solution ban kar concept clear hoga.

Payment 2 baar charge ho raha hai	→	Idempotency
Order DB mein save hua, par Kafka event miss ho gaya	→	Outbox Pattern
Payment service down hone par Order requests hang	→	Timeout + Retry + Circuit Breaker
Do customers ek saath last item order karte hain	→	Row Locking / Redis Distributed Lock
Payment fail hone par order/inventory inconsistent state mein reh jaate hain	→	Saga Pattern + Compensation

System Architecture — Bird's Eye View





Final Technology Stack

Area	Technology
Language	Python
Framework	FastAPI
Validation	Pydantic
ORM	SQLAlchemy
Migrations	Alembic
Primary Database	PostgreSQL
Search Engine	Elasticsearch / OpenSearch
Cache	Redis
Message Broker	Apache Kafka
Background Jobs	Celery
Object Storage	AWS S3 / MinIO (images, invoice PDFs)
API Gateway	Gateway layer (auth, routing, rate-limit)
Authentication	JWT / OAuth2 / OpenID Connect
Social Login (SSO)	Google, Facebook, Apple Sign-In via OAuth2
Secrets Management	HashiCorp Vault / .env + GitHub Secrets
Payment	Stripe / Razorpay (test/sandbox mode)
Email Delivery	SMTP / SendGrid / Amazon SES
SMS Gateway	Twilio / MSG91 (test/sandbox mode)
Push Notifications	Firebase Cloud Messaging (FCM)

PDF Generation	ReportLab / WeasyPrint (invoices)
Shipping Integration	Courier partner API (Shiprocket/Delhivery test mode)
API Documentation	OpenAPI / Swagger UI (auto via FastAPI)
Containers	Docker
Orchestration	Kubernetes
Metrics	Prometheus
Dashboards	Grafana
Tracing	OpenTelemetry + Jaeger
Centralized Logging	ELK Stack (Elasticsearch, Logstash, Kibana) or Loki
Testing	Pytest (unit / integration / contract / e2e)
Version Control	Git + GitHub (branching, PR review, tags)
CI/CD	GitHub Actions
Architecture Style	Microservices
Patterns	Saga, Outbox, CQRS, Circuit Breaker, Idempotency, Cache-Aside

Amazon / Flipkart Feature → Service → Tool Map

Jo bhi feature tumne real apps mein dekha hai, usko ek specific service aur tool se map kiya gaya hai — taaki koi bhi 'jaadu' na lage, sab kuchh ek engineering decision ki tarah samajh aaye.

Real feature (jo user dekhta hai)	Backend Service	Key Tool
Cart saved rehta hai chahe app band karke wapas aao	Cart Service	Redis + Postgres persistence
"Recently Viewed" / continue browsing after login	User / Session Service	Redis + Postgres
Search bar — typo-tolerant, filters, sort	Search Service	Elasticsearch / OpenSearch
"Customers also bought" / recommendations	Recommendation Service	Collaborative filtering + Redis cache
Order confirmation / shipping update email	Notification Service (Email)	SMTP / SendGrid + Celery
OTP / delivery SMS	Notification Service (SMS)	Twilio / MSG91 + Celery
App push notification (order shipped)	Notification Service (Push)	Firebase Cloud Messaging
"Download Invoice" PDF button	Invoice Service	ReportLab/WeasyPrint + S3
"Track your order" live status	Shipping Service	Courier API + status webhooks
Return / refund request flow	Return-Refund Service	Saga pattern (reverse of order flow)
Seller dashboard — add product, see own sales	Seller / Vendor Service	Multi-tenant Postgres schema
Admin dashboard — total sales, top products	Analytics / Sales Dashboard	Kafka → data pipeline → OLAP queries
"Login with Google"	Auth Service	OAuth2 / OpenID Connect (SSO)
Flash sale — limited stock, countdown timer	Coupon / Flash-Sale Service	Redis atomic counters + rate limit
Wishlist heart icon	Wishlist Service	Postgres / Redis
Product reviews & star ratings	Review & Rating Service	Postgres + moderation queue
Reward points / cashback on purchase	Loyalty Service	Postgres, updated via Kafka events
Product images load fast globally	CDN + Object Storage	S3/MinIO + CloudFront-style CDN
"1,204 people viewed this today"	Analytics Service	Redis counters + Kafka stream

Phase-Wise Learning Path

Har phase ek chhoti, complete milestone hai — pehle isko akela samjho aur code karo, tabhi agle phase par jao. Har card mein: problem jo yeh solve karta hai, seekhne ka goal, core concepts, tools, aur end deliverable.

PHASE 1

Auth Service — Foundation + SSO/OAuth2

Real-world problem

Users ko securely register/login karana hai bina password plain-text mein store kiye; aur bahut log "Login with Google" chahte hain kyunki naya account banana friction hai.

Goal

Ek standalone authentication service banana jo email/password aur social login (SSO) dono support kare, aur baaki services ke liye identity provider ki tarah kaam kare.

Core concepts

Registration, Login, JWT access + refresh token, Logout, Password hashing (bcrypt), RBAC (CUSTOMER / ADMIN / SELLER), OAuth2 Authorization Code flow, OpenID Connect, token rotation

Tools / tech

FastAPI, Pydantic, PostgreSQL, SQLAlchemy, Alembic, JWT, OAuth2 (Google/Facebook), passlib

Deliverable

Auth Service with signup/login/refresh/logout + "Login with Google" working end-to-end

PHASE 2

User Service + Session Continuity

Real-world problem

Agar user profile data Auth Service ke andar hi rahega to services tightly coupled ho jayengi; aur agar user logout ho kar wapas aaye to unka last browsing state (recently viewed, filters) gayab nahi hona chahiye.

Goal

Auth aur User data ko alag services mein rakhna (loose coupling), aur login hone par user ki last session state restore karna.

Core concepts

Service boundaries, data ownership, foreign-key-by-reference, address/preferences modelling, session persistence keyed by user_id

Tools / tech

FastAPI, PostgreSQL, SQLAlchemy, Redis (for last-session cache)

Deliverable

User Service + "Recently Viewed" list that persists across logout/login

PHASE 3

API Gateway

Real-world problem

Client ko har service ka alag URL/port pata hona bura design hai, aur auth/rate-limit har service mein alag-alag likhna duplicate kaam hai.

Goal

Ek gateway layer banana jo routing, JWT/OAuth verification, rate limiting aur logging centrally handle kare.

Core concepts

Reverse proxy routing, centralized auth check, sliding-window rate limiting, request ID propagation, CORS

Tools / tech

FastAPI / dedicated gateway layer, Redis (for rate limit)

Deliverable

Gateway routing /api/* requests to correct backend service, with auth enforced centrally

PHASE 4

Product Service + Search Service

Real-world problem	Simple SQL 'LIKE %query%' search slow aur limited hota hai — typo ('mobiel') par kuchh nahi milta, aur filters/facets (brand, price range, rating) combine karna mushkil hai.
Goal	Product catalog Postgres mein rakhna, aur ek dedicated Search Service Elasticsearch ke saath banana jo fast, fuzzy, faceted search de.
Core concepts	Inverted index, fuzzy matching, faceted search/filters, pagination, sorting, index sync from Postgres → Elasticsearch via Kafka
Tools / tech	FastAPI, PostgreSQL, Elasticsearch/OpenSearch, Kafka (for index sync)
Deliverable	Search API with typo-tolerance, filters (price/brand/rating) and pagination

PHASE 5	Cart Service (Redis-backed, persistent)
Real-world problem	Agar cart sirf frontend/localStorage mein hai, to user doosre device par login kare to cart khali milta hai — bad UX.
Goal	Server-side cart banana jo Redis mein fast rahe par Postgres mein bhi persist ho, taaki cart kabhi na khoye.
Core concepts	Cache-aside for cart, write-through persistence, cart merge on login (guest cart + saved cart)
Tools / tech	Redis, PostgreSQL
Deliverable	Cart that's identical across web/app/devices after login

PHASE 6	Wishlist & Recently Viewed Service
Real-world problem	Users apne pasand ke products baad mein dekhna chahte hain, aur unhe wahi products dikhna chahiye jo unhone recently browse kiye the.
Goal	Lightweight service banana jo wishlist add/remove aur recently-viewed tracking handle kare.
Core concepts	Time-ordered lists (Redis sorted sets), TTL-based recently-viewed expiry, deduplication
Tools / tech	Redis (sorted sets), PostgreSQL (durable wishlist)
Deliverable	Wishlist API + 'Recently Viewed' carousel data API

PHASE 7	Order Service
Real-world problem	Order banate waqt product validate karna, price calculate karna, coupon apply karna — sab ek jagah coordinate karna padta hai.
Goal	Order Service ko orchestrator ki tarah design karna jo baaki services ko call/event karke order complete kare.
Core concepts	Order state machine (PENDING → CONFIRMED → PAID → SHIPPED → DELIVERED → CANCELLED), orchestration vs choreography
Tools / tech	FastAPI, PostgreSQL, Kafka
Deliverable	End-to-end order creation flow with proper status transitions

PHASE 8	Payment Service
Real-world problem	Duplicate payment requests (network retry, double-click) double-charge kar sakte hain; webhooks ko blindly trust karna security risk hai.
Goal	Idempotent payment API banana jo webhook signature verify kare aur retries ko safely handle kare.
Core concepts	Payment intent, idempotency keys, webhook signature verification, refunds, failed-payment handling
Tools / tech	Stripe/Razorpay test mode, PostgreSQL, Redis (idempotency key store)
Deliverable	Payment Service jo same request 2 baar aane par bhi ek hi baar charge kare

PHASE 9	Inventory Service
----------------	--------------------------

Real-world problem	Do customers ek saath last available item order kar dete hain — dono ko 'order confirmed' mil jaata hai but stock sirf 1 tha.
Goal	Race-condition-free stock decrement implement karna.
Core concepts	Database row locking (SELECT FOR UPDATE), optimistic locking (version column), Redis distributed lock
Tools / tech	PostgreSQL transactions, Redis
Deliverable	Inventory API jo concurrent orders ko correctly serialize kare

PHASE 10	Coupon & Flash-Sale Service
Real-world problem	Flash sale mein 1000 log ek saath 'Buy Now' dabate hain par stock sirf 50 hai — overselling ka risk bahut high hota hai.
Goal	Coupon validation rules aur high-concurrency flash-sale stock counter banana.
Core concepts	Business rule validation (min order value, usage limit, expiry), Redis atomic decrement (DECR), rate limiting per user
Tools / tech	PostgreSQL, Redis
Deliverable	Coupon API + Flash-sale endpoint that never oversells
PHASE 11	Review & Rating Service
Real-world problem	Fake/spam reviews product trust ko kharab karte hain, aur sirf verified buyers ko hi review allow karna chahiye.
Goal	Review/rating system banana jo purchase-verification aur basic moderation ke saath ho.
Core concepts	Verified-purchase check, average rating aggregation, review moderation queue
Tools / tech	FastAPI, PostgreSQL
Deliverable	Product page ke liye reviews + average star rating API
PHASE 12	Recommendation Service
Real-world problem	Generic product listing engagement kam karti hai — personalized suggestions conversion badhate hain.
Goal	Simple collaborative-filtering based recommendation engine banana, order history se train karke.
Core concepts	Collaborative filtering basics, precomputed recommendation cache, cold-start problem
Tools / tech	Python (pandas/numpy for offline job), Redis (serve cached recommendations)
Deliverable	"Related products" API powered by real purchase-pattern data
PHASE 13	Shipping & Logistics Service
Real-world problem	Order place hone ke baad customer ko real-time pata hona chahiye ki package kahan hai.
Goal	Courier partner ke sandbox API ko integrate karna aur shipment status ko order ke saath sync rakhna.
Core concepts	Third-party API integration, status webhooks, address validation
Tools / tech	Courier API (Shiprocket/Delhivery test mode), Kafka (status events)
Deliverable	"Track Order" API showing live shipment status
PHASE 14	Return & Refund Service
Real-world problem	Return request aane par payment refund karna, inventory wapas badhana, aur order status update karna — sab consistent hona chahiye.
Goal	Return/refund ko Saga pattern se model karna, jaisa forward order flow tha.
Core concepts	Compensating transactions, refund-to-source, partial returns
Tools / tech	Kafka, Payment Service, Inventory Service
Deliverable	Return request flow jo refund + stock restore + status update karta ho

PHASE 15	Seller / Vendor Service
Real-world problem	Ek hi platform par multiple sellers apne products list karte hain aur sirf apna data/sales dekh sakne chahiye.
Goal	Multi-tenant seller service banana jahan har seller sirf apna data access kare.
Core concepts	Multi-tenancy, seller onboarding, per-seller product ownership, seller-level RBAC
Tools / tech	FastAPI, PostgreSQL (tenant_id scoping)
Deliverable	Seller dashboard API: add product, view own orders, view own sales

PHASE 16	Invoice Service
Real-world problem	Order complete hone ke baad customer ko GST/tax details ke saath ek downloadable invoice chahiye.
Goal	Order data se automatically PDF invoice generate karna aur secure download link dena.
Core concepts	Server-side PDF generation, object storage upload, signed/expiring download URLs
Tools / tech	ReportLab/WeasyPrint, S3/MinIO, Celery (async generation)
Deliverable	"Download Invoice" button jo real PDF return kare

PHASE 17	Notification Service — Email, SMS, Push
Real-world problem	Order confirm/shipped/delivered par customer ko email, OTP ke liye SMS, aur app open na ho tab bhi push notification chahiye.
Goal	Ek unified Notification Service banana jo Kafka events sunkar sahi channel (email/SMS/push) choose kare.
Core concepts	Event-triggered notifications, template rendering, channel fallback (push fail → SMS), delivery status tracking
Tools / tech	SMTP/SendGrid (email), Twilio/MSG91 (SMS), Firebase Cloud Messaging (push), Celery
Deliverable	Order lifecycle ke har stage par sahi channel par notification jaana

PHASE 18	Redis — Beyond Caching
Real-world problem	Product reads database ko slow kar rahe hain; APIs abuse ho sakti hain; concurrent stock updates race conditions face kar rahe hain.
Goal	Redis ko production ke real use-cases mein systematically lagana.
Core concepts	Cache-aside pattern, cache invalidation, sliding-window rate limiting, distributed locks, session/OTP storage
Tools / tech	Redis
Deliverable	Product cache + rate limiter + inventory lock + cart, sab Redis se powered

PHASE 19**Kafka — Event Backbone****Real-world problem**

Order create hone par Inventory, Payment, Notification, Analytics services ko ek-doosre ko directly call karna tightly coupled aur fragile hai.

Goal

Event-driven communication seekhna — services ek-doosre ko event ke through inform karein, direct call nahi.

Core concepts

Topic, Partition, Offset, Producer/Consumer, Consumer group, Retry, Dead Letter Queue, Event schema, Idempotent consumer

Tools / tech

Apache Kafka

Deliverable

OrderCreated / PaymentCompleted / ReturnRequested events publish + consume across 6+ services

PHASE 20**Celery — Background Jobs****Real-world problem**

Email bhejna, invoice generate karna, reports banana, expired-cart cleanup — yeh sab request ko slow nahi karna chahiye.

Goal

Async background job processing seekhna aur Kafka (events between services) vs Celery (async tasks within a service) ka fark practically samajhna.

Core concepts

Task queue, worker, broker (Redis/RabbitMQ), scheduled/periodic tasks (Celery Beat)

Tools / tech

Celery, Redis/RabbitMQ as broker

Deliverable

Notification, Invoice aur Report generation sab Celery tasks ke through async chal rahe hon

PHASE 21**Admin & Sales Analytics Dashboard****Real-world problem**

Business ko real-time decisions ke liye sales/traffic data chahiye, par har query directly production Order DB par nahi maar sakte — usse app slow ho jayega.

Goal

Kafka events se ek separate analytics pipeline banana jo dashboards ko power kare bina main DB load kiye.

Core concepts

Event stream → aggregation, OLAP-style queries, read-model separate from write-model (CQRS)

Tools / tech

Kafka, PostgreSQL (analytics schema) or a columnar store, scheduled aggregation jobs

Deliverable

Admin dashboard API: today's sales, top-selling products, revenue by category

PHASE 22**Resilience & Consistency Patterns****Real-world problem**

Order DB mein save ho gaya par Kafka event chala nahi gaya; payment fail hone par order/inventory inconsistent reh jaate hain; payment service down hone par requests hang ho jaati hain.

Goal

Distributed transactions aur partial-failure scenarios ko gracefully handle karna seekhna.

Core concepts

Saga Pattern + compensation, Outbox Pattern, Timeout, Exponential Backoff Retry, Circuit Breaker, Fallback

Tools / tech

PostgreSQL transactions, Kafka, custom resilience logic

Deliverable

Order flow jo payment failure par safely rollback/compensate kare

PHASE 23**Docker****Real-world problem**

20+ services ko manually alag terminal mein chalana aur dependencies (Postgres, Redis, Kafka, Elasticsearch) manage karna practically impossible hai.

Goal

Har service ko containerize karna aur ek command se poora system chalana.

Core concepts

Dockerfile, multi-container apps, networking between containers, volumes, environment-based config

Tools / tech

Docker, Docker Compose

Deliverable

docker-compose.yml jo pura ShopSphere stack ek command se start kare

PHASE 24**Observability, Security, Kubernetes & CI/CD****Real-world problem**

Ek request Gateway → Order → Payment → Kafka → Notification tak jaata hai — agar slow/insecure ho to production mein pata chalna chahiye, aur deployment automated honi chahiye.

Goal

Distributed tracing, security hardening, container orchestration aur automated pipeline — sab ek saath production-ready banana.

Core concepts

Trace ID propagation, structured logging, OWASP basics, Pod/Deployment/Service/Ingress/HPA, Rolling Deployment, CI/CD pipeline

Tools / tech

OpenTelemetry, Prometheus, Grafana, Jaeger, Kubernetes, GitHub Actions

Deliverable

ShopSphere running on Kubernetes with monitoring dashboards and an automated deploy pipeline

Git & Version Control — Choti se Choti Cheez

20+ services likhte waqt Git sirf 'git add, commit, push' tak seem ■■■ nahi rehta — industry mein branching, commit conventions aur release tagging bhi utna hi important hai jitna khud ka code.

Topic	Kya seekhna hai
Repo strategy	Monorepo (sab services ek repo mein, alag folders) vs Polyrepo (har service apna repo) — ShopSphere ke liye monorepo se shuru karo, seekhna simple hai
Branching model	main (production-ready) + develop + feature/xyz branches — ya trunk-based with short-lived feature branches
Commit convention	Conventional Commits: feat:, fix:, chore:, refactor:, test:, docs: — isse changelog auto-generate ho sakta hai
Pull Requests	Har feature ek PR ke through main mein merge ho, kam se kam 1 review ke baad
.gitignore	.env, __pycache__, .venv, node_modules jaise files ko commit hone se rokna
Git hooks	pre-commit hook se lint/format/tests automatically chalwana commit se pehle
Semantic Versioning	MAJOR.MINOR.PATCH (e.g. v1.4.2) — har service ka apna version tag
Git tags & releases	Ek stable point par git tag v1.0.0 lagana, GitHub Release banana
Secrets safety	API keys/passwords kabhi commit nahi karna — .env + git-secrets/pre-commit scan use karna
Rebase vs Merge	Feature branch ko clean history ke liye rebase karna, shared branch ko merge karna
Code review checklist	Naming, tests included, no secrets, no debug prints, migration included agar schema change hai

Security Checklist — Har Service Mein Follow Karna

- Passwords kabhi plain-text mein store na ho — hamesha bcrypt/argon2 se hash karo.
- JWT access token ka expiry chhota rakho (15-30 min), refresh token rotate karo.
- Har input ko validate karo (Pydantic schemas) — SQL Injection se ORM parameterized queries bachaati hain.
- CORS ko sirf trusted origins ke liye enable karo, wildcard (*) production mein mat use karo.
- Rate limiting har public endpoint par lagao, especially login aur OTP APIs par.
- Payment webhooks ki signature hamesha verify karo — bina verification trust mat karo.
- Secrets (.env, API keys) ko kabhi Git mein commit mat karo — Vault ya GitHub Secrets use karo.
- Har jagah HTTPS/TLS use karo, especially payment aur auth endpoints par.
- Dependencies ko regularly scan karo (Dependabot / pip-audit) — outdated libraries vulnerabilities laati hain.
- User data (address, phone) encrypt karo at rest jahan possible ho — basic GDPR-style data privacy soch ke saath design karo.

Testing Strategy

Layer	What it checks
Unit Tests	Individual function/class logic in isolation (pytest)
Integration Tests	Service + real DB/Redis interaction
API Tests	Full request/response contract of an endpoint
Contract Tests	Ensures Payment Service API doesn't break Order Service's expectations
End-to-End Tests	Full user journey: signup → order → payment → notification

Setup Note

Current machine par Docker virtualization issue hai — isliye Phase 23 (Docker) tak architecture aur code development ho sakta hai; jab Docker-compatible environment available ho, tabhi containerization practically run karke test karna.

How to Use This Roadmap

- Ek time par sirf ek phase par focus karo — agla phase tabhi start karo jab current phase ka code chal raha ho.
- Har phase shuru karne se pehle 'Real-world problem' padho — pehle problem samjho, phir solution/tool seekho.
- Jab bhi naya concept aaye, khud ek chhota experiment likh kar dekho (e.g. do parallel requests bhej kar race condition reproduce karo).
- Docker tak pahunchne ke baad, agar environment available ho to poora stack docker-compose se chhalao.
- Observability aur Testing ko ek baar Phase 5-6 ke baad hi halka-halka introduce karna shuru kar sakte ho, taaki habit ban jaye.